

PLATFORM ENGINEER

직접 굴러보며 이해하고, 반복되는 불편을 도구로 바꿉니다.

서버와 네트워크를 직접 구성하고 서비스를 운영하며 생긴 문제를 끝까지 따라잡니다. 한 번 해결한 불편은 다시 사람의 주위에 맡기지 않도록 자동화하려 합니다.

- 01

Heimdall

저장소 등록부터 검증된 Preview 전환까지 반복 배포 절차를 자동화

DEPLOYMENT
- 02

Gjallar

Proxmox 변경을 승인·중복 방지·사후 확인으로 통제하는 운영 도구

INFRASTRUCTURE
- 03

K-Le-PaaS

자연어 요청을 제한된 Kubernetes 실행 계획과 피드백으로 연결

KUBERNETES

조운호 · Platform Engineer

[github.com/CodingPenguin-yoon](#)
[code.penguin.yoon@gmail.com](#)
[yoonman.page](#)

HOW THE WORK EVOLVED

궁금한 것을 직접 확인하는 과정에서 운영 자동화를 시작했습니다.

홈랩에 서비스를 추가할 때마다 VM 사양과 IP를 다시 정했고, 수동 주소 관리 중 IP 충돌도 겪었습니다. K-Le-PaaS에서 요청을 계획과 실행 단위로 나누는 경험을 한 뒤 반복 절차를 도구로 만들기 시작했습니다.

01 · LEARN TO STRUCTURE

K-Le-PaaS

입력 → 계획 → 확인 → 실행 → 피드백으로 운영 요청을 구조화했습니다.



02 · REDEFINE THE SCOPE

Heimdall

애플리케이션 배포와 Preview 전환

Gjallar

Proxmox 관찰과 제한된 변경 통제

VALIDATION ENVIRONMENT

직접 운영하는 환경에서 확인합니다.

Proxmox 3노드와 IPFire로 분리한 RED·GREEN·ORANGE 네트워크, NAS/NFS, WireGuard·OCI reverse proxy 환경에서 정상 경로와 실패 경로를 검증합니다.

Proxmox VE 3-node

IPFire segmentation

NAS / NFS

WireGuard / OCI

CORE SKILLS

Platform	Linux · Docker · Kubernetes · Proxmox
Backend	Python · FastAPI · PostgreSQL · REST API
Delivery	Git · NGINX · GitHub Actions · Prometheus
Network	IPFire · WireGuard · Reverse Proxy · NFS

WHAT CHANGED IN MY APPROACH

VM 생성과 애플리케이션 배포를 한 흐름으로 묶으려다 성공 조건과 복구 범위가 다르다는 것을 확인했고, Gjallar와 Heimdall로 책임을 나눴습니다.

01

HEIMDALL

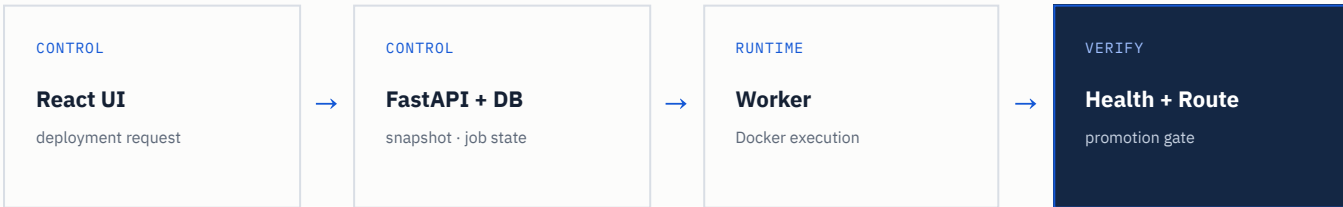
APPLICATION DELIVERY AUTOMATION · PERSONAL PROJECT

VM 생성부터 배포까지 한 번에 묶으려다, 책임을 다시 나눴습니다.

처음에는 Proxmox VM 생성과 애플리케이션 배포를 하나의 흐름으로 만들려 했습니다. 구현과 시험 과정에서 두 작업의 성공 조건과 실패 시 보존 범위가 다르다는 것을 확인했습니다.

PROBLEM	FIRST ATTEMPT	TURNING POINT
애플리케이션마다 실행 환경과 공개 경로를 반복해서 준비해야 했습니다. 저장소 checkout과 Docker build·실행, 환경 설정, 프로젝트 DB, NGINX route를 애플리케이션마다 다시 연결하고 결과를 별도로 확인해야 했습니다.	Proxmox template clone부터 애플리케이션 배포까지 잇는 통합 프로토타입을 만들었습니다. VM을 만들고 CPU·메모리를 조정한 뒤 SSH 준비, 애플리케이션 전달과 실행까지 연결하려 했습니다. 하지만 이 통합 흐름을 end-to-end로 완성하기 전에 제어 대상의 차이를 발견했습니다.	VM 생성과 애플리케이션 배포의 권한·실패 범위·완료 조건이 달랐습니다. 완성되지 않은 통합을 밀어붙이지 않고 VM 운영은 Gjallar로, 애플리케이션 배포는 Heimdall로 분리했습니다. Heimdall은 실행 중인 공용 플랫폼의 배포와 Preview 전환에 집중했습니다.

BEFORE	AFTER
애플리케이션마다 VM·Docker·공개 경로를 수동 준비 새 버전의 실행 여부와 실제 서비스 응답을 별도로 확인	실행 중인 공용 플랫폼에 저장소와 설정을 등록 candidate 검증과 Preview 전환을 하나의 배포 흐름으로 수행

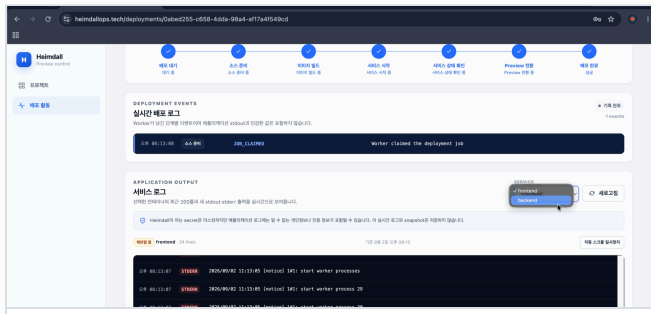


AUTOMATED DELIVERY WITH A SAFETY GATE · RESULT

실패한 candidate가 기존 서비스를 건드리지 않는지 확인했습니다.

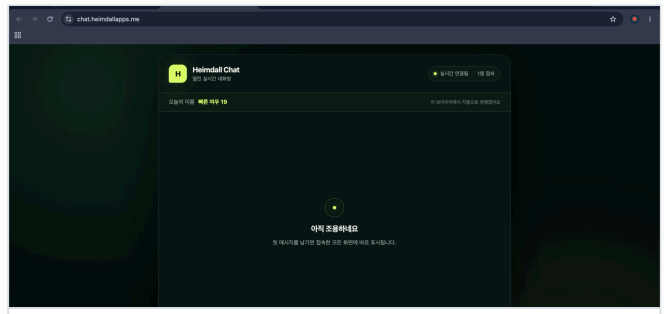
책임을 나눈 뒤 반복 배포 절차를 하나의 흐름으로 자동화했습니다. 현재 범위는 Public GitHub·fixed main·single Docker host이며, 배포 대상을 고정한 뒤 검증을 통과한 candidate만 활성화합니다.

01	02	03	04	05
PIN	BUILD	CHECK	ROUTE	PROMOTE
exact SHA와 설정 snapshot 고정	분리된 Docker candidate 빌드	서비스별 health 확인	NGINX 적용 후 endpoint probe	검증된 Preview만 활성화



LIVE DEPLOYMENT

단계별 진행 상태와 서비스 로그를 함께 확인



LIVE PREVIEW

Frontend·Backend·PostgreSQL을 연결한 실제 서비스

FAILURE TEST

Backend 시작 실패를 주입해 Preview 전환 차단을 확인했습니다.

3-tier 테스트 애플리케이션의 새 candidate에서 Backend 시작 실패를 발생시켰습니다. Health 검증 실패로 전환이 중단됐고, 기존 stable Preview는 같은 URL에서 계속 응답했습니다.



RUNTIME	VERIFY	CURRENT SCOPE / WORKING RULE
Frontend + Backend + Project PostgreSQL	service health + NGINX route probe	실패했을 때 지켜야 할 상태를 먼저 정의

[2-minute demo ↗](#)[Repository ↗](#)[Gateway tests ↗](#)[Runtime smoke ↗](#)

02

GJALLAR

CONTROLLED INFRASTRUCTURE OPERATIONS · PERSONAL PROJECT

IP 충돌을 겪은 뒤, VM 변경을 사람의 기억에만 맡기지 않기로 했습니다.

홈랩의 VM 사양과 주소를 직접 관리하며 겪은 반복과 실수를 줄이기 위해, Proxmox의 실제 상태를 읽고 제한된 변경을 통제하는 도구를 만들었습니다.

PROBLEM	DESIGN RISK	SOLUTION
VM마다 사양과 IP를 다시 정했고, 수동으로 주소를 관리하다 IP가 충돌했습니다. 같은 용도의 VM도 vCPU·메모리·디스크를 반복해서 입력했습니다. 작업 전에 live inventory를 확인하지 않으면 같은 실수를 사람의 주의로만 막아야 했습니다.	요청 수락과 목표 상태 도달은 서로 다른 결과입니다. 비동기 작업은 timeout이나 task reference 누락, 완료 후 상태 불일치가 생길 수 있다고 보고 API 응답과 운영 결과를 분리해 확인하도록 설계했습니다.	사양을 재사용하고, 변경 전후의 실제 상태를 확인하도록 했습니다. live inventory를 기준으로 권한 ·IP·network·storage를 확인하고, 승인된 작업만 실행합니다. 전체 VM lifecycle 자동화보다 안전한 실행 경계를 우선했습니다.

SOURCE OF TRUTH		OPERATION RECORD
Proxmox VE Node / VM inventory Task reference and status Observed resource state	READ LIVE STATE ↔ EXECUTE EXPLICIT CHANGE	Gjallar Operator intent and policy Approval · idempotency Post-check and reconciliation

BEFORE	AFTER
VM마다 사양·IP 조건을 다시 확인	Profile 재사용 + live preflight로 잘못된 요청 차단

GUARDED EXECUTION

변경 작업을 같은 버튼이 아니라, 같은 검증 원칙으로 묶었습니다.

현재 구현한 Create·Start·graceful Shutdown은 사전 확인·실행·task 추적·실제 상태 재조회로 끝납니다.

Guided unlock은 운영자 실행 후 해제 상태를 다시 확인합니다.

01 PLAN live inventory로 사전 조건 계산	02 CONFIRM 권한·승인·위험 확인	03 EXECUTE 제한된 Proxmox API 변경
04 TRACK UPID task 완료 상태 확인	05 POST-CHECK 변경 후 실제 VM 상태 조회	06 RECORD 성공 또는 재확인 상태 기록

01 CREATE VM Profile · IP preflight · 승인 · UPID · 생성 후 상태 확인	02 START 명시적 확인 · idempotency · task polling · running 확인
03 GRACEFUL SHUTDOWN 허용 상태 확인 · 제한된 실행 · 종료 상태 재조회	04 GUIDED UNLOCK 짧은 지침 발급 · 운영자 실행 · API로 lock 해제 확인

DESIGNED NON-SUCCESS PATH		
TIMEOUT 응답 지연을 자동 성공으로 변경하지 않음	MISSING TASK task reference가 없으면 결과를 확정하지 않음	STATE MISMATCH 예상 상태와 다른 재확인 대상으로 기록

RESULT 반복 입력을 줄이고, 변경 결과를 실제 상태로 다시 확인했습니다. VM Profile로 자원 사양을 재사용하고 IP 충돌을 사전에 확인합니다. Create·Start·graceful Shutdown의 task와 after-state를 검증했고, guided unlock은 운영자 실행 후 lock 해제를 재조회합니다.	WHAT CHANGED 자동으로 실행해도 되는 범위와 운영자가 판단해야 하는 범위를 먼저 나누게 됐습니다. Create·Start·graceful Shutdown과 제한된 guided unlock을 승인 흐름으로 제공합니다. migration·snapshot·임의 shell 작업은 현재 실행 범위에 포함하지 않았습니다.
---	--

github.com/CodingPenguin-yeon/Gjallar

03

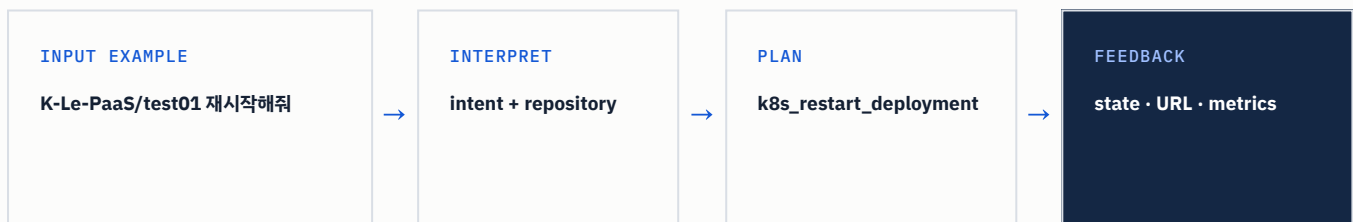
K-LE-PAAS

KUBERNETES OPERATIONS · TEAM OF 2 · 2025.09-12

자연어 운영 요청을 실행하기 전에, 검증 가능한 계획으로 바꿨습니다.

2인 팀에서 자연어 요구 변환과 인프라 모니터링을 맡았습니다. 웹과 Slack의 요청을 제한된 실행 계획으로 바꾸고 결과·지표·접근 URL을 다시 사용자에게 연결했습니다.

PROBLEM	MY ROLE	TEAM DECISION
자연어를 그대로 실행하면 해석 오류가 운영 변경으로 이어질 수 있습니다. 모호한 요청에서 대상과 의도를 분리하고, 시스템이 지원하는 작업과 인자만 실행하도록 제한할 필요가 있었습니다.	요청을 CommandRequest와 허용된 CommandPlan으로 구조화했습니다. 상태·로그 조회, 재시작·스케일링·버전 롤백 API로 연결하고 실행 결과와 외부 URL을 다시 사용자에게 반환했습니다.	공모전 조건과 개발 기간을 기준으로 NCP 배포 흐름에 합의했습니다. 저는 온프레미스 배포와 NCP 게이트웨이를, 팀원은 NCP 중심 구성을 제안했습니다. 공동 목표와 제약을 비교해 NCP SourcePipeline과 NKS를 사용하고 담당 기능의 완성도에 집중했습니다.



INITIAL OPTION	WHY WE CHANGED
내가 제안한 온프레미스 배포 + NCP 게이트웨이 직접 운영 환경을 활용하는 구성을 제안	NCP SourcePipeline → NKS 공모전 조건·개발 기간·출일 수 있는 구축 범위를 함께 비교

FROM REQUEST TO FEEDBACK

요청을 실행하고, 상태·URL·지표를 다시 사용자에게 돌려줬습니다.

자연어를 자유 형식 명령으로 실행하지 않고, 검증 가능한 작업 단위로 변환해 Kubernetes API와 NKS 관측 경로에 연결했습니다.

01	02	03	04	05
PARSE	VALIDATE	PLAN	EXECUTE	FEEDBACK
자연어를 command-parameters JSON으로 변환	CommandRequest로 형식과 대상 검증	허용된 CommandPlan(tool, args) 생성	Kubernetes Python Client API 호출	상태·URL·메트릭을 사용자에게 반환

MY CONTRIBUTION

자연어 실행 계획

command-parameters 구조화와 Kubernetes 실행·조회 연결

NKS 모니터링

Prometheus 설치·수집 대상 구성, CPU·메모리·디스크·네트워크 조회 API

서비스 접근 경로

Ingress 외부 주소와 사용자·저장소별 service URL 생성·저장·조회

WHAT CHANGED

- 자유 형식 요청보다 검증 가능한 중간 표현을 먼저 만듭니다.
- 기술 선택은 개인 취향보다 프로젝트의 제약으로 결정합니다.
- 실행 결과를 사용자가 확인할 피드백까지 연결합니다.

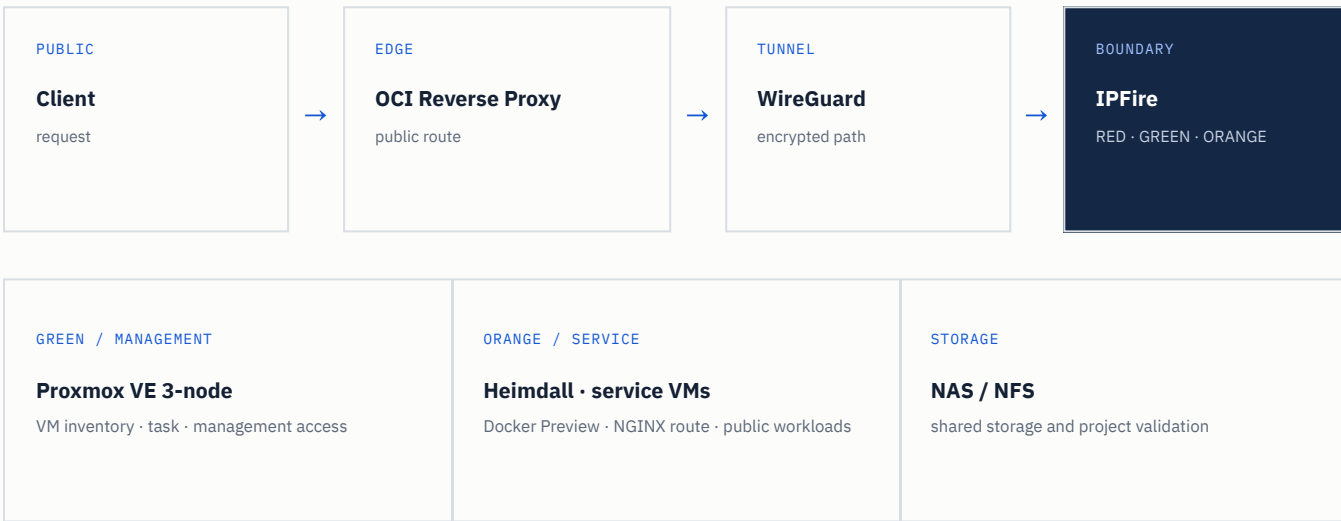
RESTART TARGET	API ACTION	AFTER STATE
owner / repository → Deployment	patch_namespaced_deployment()	state · metrics · service URL
저장소 식별자를 실행 대상 namespace와 Deployment로 매핑	pod template annotation을 변경해 rollout restart 수행	실행 이후 확인에 필요한 운영 정보를 요청 흐름으로 반환

DELIVERY RECORD	RUNTIME RESULT	DEMO
Backend 27 + Frontend 21 · merged PR 48개 (authored)	NKS 작업·모니터링·Ingress URL 흐름 시연	12-minute walkthrough ↗

HOME LAB FAILURE CASE

직접 구성하고, 문제가 생기면 흐름을 나눠 확인합니다.

망분리 후 외부 서비스가 끊겼을 때 보안 경계를 되돌리지 않고 요청과 반환 경로를 나눠 단절 구간을 좁혔습니다.
필요한 경로를 복구하면서 ORANGE에서 GREEN 관리망으로의 접근 차단은 유지했습니다.



HOW I WORK NOW

- 직접 확인** 궁금한 것은 직접 구성하고, 문제를 흐름과 경계로 나눠 확인합니다.
- 도구로 남기기** 한 번 해결한 과정은 다음에 다시 사용할 수 있는 도구로 남깁니다.
- 작게 완성하기** 범위가 커지면 한 흐름부터 완성하며, 인프라와 애플리케이션 사이의 반복을 줄이고 재사용할 도구를 더 깊게 만들고 싶습니다.

EDUCATION & CERTIFICATIONS

광운대학교 전자통신공학과 · 2026.02 졸업

정보처리기사 · 리눅스마스터 2급

CONTACT

조윤호 · Platform Engineer

code.penguin.yoon@gmail.com
github.com/CodingPenguin-yoon
[yoonman.page](#)